DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute affiliated to VTU, Belagavi, Approved by AICTE & ISO 9001:2008 Certified)
Accredited by National Assessment & Accreditation Council (NAAC) with 'A' grade,
Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru-560111

Mini-Project Report

on

"CINESENTIMENT: A NEAR-REAL-TIME MOVIE REVIEW SENTIMENT ANALYTICS PLATFORM USING KAFKA, HDFS AND LOGISTIC REGRESSION"

Submitted by

Supreeth C (1DS23CS227)
Vaibhav Garg (1DS23CS245)
Thejas Sreejesh (1DS23CS237)
Vaibhav Srivastava (1DS23CS246)

Seventh Semester B.E.(CSE)

2026-2027

Under the guidance of

Prof. Nidhishree
Assistant Professor
Dept. of CSE
DSCE, Bangalore

Department of Computer Science and Engineering
Dayananda Sagar College of Engineering
Bangalore-560111

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

DAYANANDA SAGAR COLLEGE OF ENGINEERING

Shavige Malleshwara Hills, Kumaraswamy Layout, Bangalore - 560111

Department of Computer Science & Engineering

CERTIFICATE

This is to certify that the mini-project entitled **CineSentiment: A Near-Real-Time Movie Review Sentiment Analytics Platform using Kafka, HDFS and Logistic Regression** is a bonafide work carried out by **Supreeth C [1DS23CS227], Vaibhav Garg [1DS23CS245], Thejas Sreejesh [1DS23CS237], Vaibhav Srivastava [1DS23CS246]** in partial fulfilment of 7th semester, Bachelor of Engineering in Computer Science and Engineering under Visvesvaraya Technological University, Belgaum during the year 2026-27.

Prof. Nidhishree	**Dr. Ramesh Babu D R**	**Dr. B G Prasad**
(Internal Guide)	Vice principal & Head	Principal, DSCE
Assistant Professor,	Department of CSE,DSCE	
Department of CSE,DSCE		
Signature.....	Signature.....	Signature.....

Name of the Examiners:	Signature with date:
1.	.....
2.	.....

ACKNOWLEDGEMENT

We are pleased to have successfully completed the Mini project **"CineSentiment: A Near-Real-Time Movie Review Sentiment Analytics Platform using Kafka, HDFS and Logistic Regression"**. We thoroughly enjoyed the process of working on this project and gained a lot of knowledge doing so.

We would like to take this opportunity to express our gratitude to **Dr. B G Prasad**, Principal of DSCE, for permitting us to utilize all the necessary facilities of the institution. We also thank our respected Vice Principal, HOD of Computer Science & Engineering, DSCE, Bangalore, **Dr. RameshBabu D R**, for his support and encouragement throughout the process. We are immensely grateful to our respected and learned guide, **Prof. Nidhishree**, Assistant Professor CSE, DSCE for his/her valuable help and guidance. We are indebted to them for their invaluable guidance throughout the process and their useful inputs at all stages of the process.

Supreeth C, 1DS23CS227
Vaibhav Garg, 1DS23CS245
Thejas Sreejesh, 1DS23CS237
Vaibhav Srivastava, 1DS23CS246

CONTENTS

SL. NO	CONTENT	PG. NO
1.	Abstract	3
2.	Introduction	4-5
3.	Literature Survey	6-12
4.	System design & Methodology	13-17
5.	Snapshots and Results	18-20
6.	Conclusion & Future Enhancements	21
7.	References	22-23
8.	Appendix	24

ABSTRACT

Studios and streaming platforms need to know how audiences react to a film within hours of its release, but sentiment reporting is still largely manual and arrives days late. This project presents CineSentiment, a near-real-time sentiment analytics platform for movie reviews built on a big data stack. Reviews are published to **Apache Kafka**, persisted as raw and scored micro-batches on the **Hadoop Distributed File System (HDFS)**, and classified as positive or negative by a **logistic regression model trained on TF-IDF features** of the IMDB 50K review dataset. A stream processor maintains a running average sentiment score for every movie title, which is served through a REST API to an interactive web dashboard with a poster library, per-title trends, model analysis and a review-submission page. On 10,000 held-out reviews the classifier reaches 89.39% accuracy and a ROC-AUC of 0.9606. The complete pipeline is containerised with Docker Compose and also runs in a mock mode without a cluster.

Chapter 1

INTRODUCTION

Every film release now generates a continuous stream of audience opinion in the form of reviews, ratings and social media posts. For studios and streaming platforms this feedback is commercially valuable: it guides marketing spend, recommendation ranking and decisions about sequels. In practice, however, sentiment reporting is often compiled manually from sampled reviews and reaches decision makers days after the opinions were written, by which time the release window has largely passed. The volume, velocity and unstructured nature of review text make this a natural big data analytics problem.

This project addresses the problem with an end-to-end streaming pipeline. Movie reviews are ingested through Apache Kafka, stored on HDFS for durability and later batch analysis, and scored by a logistic regression classifier over TF-IDF features that labels each review as positive or negative and produces the probability of positive sentiment. A running average of this score is maintained for every movie title, so that the audience reaction to a film is visible as it evolves. The results are exposed through a FastAPI service and a web interface consisting of a home page, a movie library with posters, a live dashboard, an analysis page and a page where a user can write a review for any movie and instantly see its sentiment.

In this report we describe the motivation, the related research, the architecture and methodology of the system, and the conclusions and future enhancements. The Kaggle IMDB Dataset of 50,000 movie reviews is used for training and for replaying a live stream, a poster archive with IMDb metadata powers the movie library, and the entire stack is packaged with Docker Compose so that it can be reproduced on a single machine.

Six key introductory points summarizing the **core aspects** of the **sentiment analytics** project:

- **Streaming Ingestion with Kafka**: Reviews are published to a partitioned Kafka topic keyed by movie title, decoupling producers from consumers and allowing the pipeline to absorb bursts of reviews after a release.

- **Durable Storage on HDFS**: Raw and scored reviews are written to HDFS as time-partitioned JSON-lines part files, giving a replicated, immutable record that supports reprocessing and batch retraining.

- **TF-IDF and Logistic Regression**: A lightweight, interpretable classifier turns each review into a probability of positive sentiment and reaches 89.39% accuracy on held-out data.

- **Running Sentiment per Title**: The stream processor updates a running average score, positive share and review count for each movie after every review, replacing manual, delayed reporting.

- **Interactive Dashboard and Library**: A multi-page web interface shows live pipeline health, per-title trends, a poster library with IMDb ratings, model analysis, and a page to submit reviews for any movie.

- **Reproducible and Portable**: Docker Compose starts Kafka, HDFS, the processor, the API and the web server together, and a mock mode replaces Kafka and HDFS with local files when no cluster is available.

Chapter 2

LITERATURE SURVEY

The literature review examines research on the three building blocks of this project: machine-learning sentiment classification of movie reviews, distributed log and file systems for ingesting and storing large volumes of data, and stream-processing models for low-latency analytics. The purpose of the survey is to understand what accuracy simple models can achieve on review text, which architectural patterns are used to move and store data at scale, and where gaps remain that an integrated platform can fill.

1. **Title: Thumbs up? Sentiment Classification using Machine Learning Techniques | Authors: Bo Pang, Lillian Lee, Shivakumar Vaithyanathan | Year: 2002 | Venue: Proceedings of EMNLP 2002, pp. 79-86 | Affiliation: Cornell University and IBM Almaden Research Center, USA**

   Abstract: The paper treats sentiment analysis of movie reviews as a text-classification problem and compares Naive Bayes, maximum entropy and support vector machines against keyword baselines proposed by human judges, using reviews collected from the IMDb archive.

   Key Points:

   - **Machine Learning vs. Heuristics**: All three learning algorithms clearly outperform hand-built keyword baselines, with accuracies in the range of roughly 77-83%.

   - **Feature Choice**: Unigram presence features work better than term frequency, an observation that motivates weighted bag-of-words representations such as TF-IDF.

   - **Harder than Topic Classification**: Sentiment is subtler than topic, which explains the lower accuracy compared with conventional text categorisation.

Drawbacks of the Report:

- **Small Dataset**: Only about 1,400 reviews were used, so conclusions may not transfer to large corpora.

- **Offline and Batch Only**: The study does not consider streaming data, storage or deployment.

- **No Aggregation**: Reviews are classified individually; no per-movie running score is produced.

2. **Learning Word Vectors for Sentiment Analysis | Authors: Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, Christopher Potts | Year: 2011 | Venue: Proceedings of ACL-HLT 2011, pp. 142-150 | Affiliation: Stanford University, USA**

   Abstract: The paper introduces the Large Movie Review Dataset of 50,000 labelled IMDB reviews and a model that learns word vectors capturing both semantic and sentiment information, combined with bag-of-words features.

   Key Points:

   - **Benchmark Dataset**: A balanced set of 25,000 training and 25,000 test reviews with polar labels, and with disjoint movies in the two splits; this is the dataset used in our project.

   - **Accuracy**: The best model reaches close to 89% accuracy, a figure our simple TF-IDF and logistic regression pipeline matches (89.39%).

   - **Unsupervised plus Supervised Learning**: Unlabelled reviews are used to learn vectors, which improves over bag-of-words alone.

Drawbacks of the Report:

- **Static Benchmark**: The work is evaluated offline and does not address real-time scoring or deployment.

- **Training Cost**: Learning word vectors requires a more complex training procedure than a TF-IDF pipeline.

- **Opaque Features**: Dense vectors are harder to interpret than per-word coefficients.

3. **Baselines and Bigrams: Simple, Good Sentiment and Topic Classification | Authors: Sida Wang, Christopher D. Manning | Year: 2012 | Venue: Proceedings of ACL 2012 (Short Papers), pp. 90-94 | Affiliation: Stanford University, USA**

   Abstract: The paper shows that carefully designed linear models over unigram and bigram features, including Naive Bayes log-count-ratio features fed to an SVM (NBSVM), are strong baselines for sentiment and topic classification.

   Key Points:

   - **Strong Linear Baselines**: Simple linear classifiers over n-gram features are competitive with far more complex models on the IMDB review data.
   - **Feature Weighting Matters**: How word counts are weighted often matters more than the choice of classifier, which supports the use of TF-IDF with logistic regression.
   - **Efficiency**: Linear models train quickly and score reviews with very low latency.

Drawbacks of the Report:

- **Higher Dimensionality**: Bigram features enlarge the feature space and memory requirements.
- **Offline Evaluation**: Streaming, storage and serving concerns are outside the scope of the paper.

4. **Kafka: A Distributed Messaging System for Log Processing | Authors: Jay Kreps, Neha Narkhede, Jun Rao | Year: 2011 | Venue: Proceedings of the NetDB Workshop | Affiliation: LinkedIn Corp., USA**

   Abstract: The paper presents Apache Kafka, a distributed publish-subscribe system designed to collect and deliver large volumes of log data with low latency, combining ideas from log aggregators and messaging systems.

Key Points:

- **Partitioned, Append-Only Logs**: Topics are split into partitions stored as sequential logs, giving high throughput and ordered delivery within a partition.
- **Pull-Based Consumers and Offsets**: Consumers track their own offsets and can re-read data, which allows replay and at-least-once processing.
- **Consumer Groups**: Several consumers can share a topic, enabling horizontal scaling of processing.

Drawbacks of the Report:

- **Coordination Dependency**: The original design relied on ZooKeeper; modern Kafka replaces it with KRaft, which our deployment uses.
- **Transport Only**: Kafka moves data but does not analyse it; classification and aggregation must be built on top.

5. **The Hadoop Distributed File System | Authors: Konstantin Shvachko, Hairong Kuang, Sanjay Radia, Robert Chansler | Year: 2010 | Venue: IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST), pp. 1-10 | Affiliation: Yahoo!, Sunnyvale, USA**

Abstract: The paper describes the architecture of HDFS, the storage layer of Hadoop, designed to store very large data sets reliably on commodity hardware and to stream them at high bandwidth to analysis applications.

Key Points:

- **NameNode and DataNodes**: A NameNode holds the file-system metadata while DataNodes store replicated blocks.
- **Write-Once, Read-Many**: Files are appended or written once and read many times, which suits logs and scored-event archives.
- **Replication and Fault Tolerance**: Blocks are replicated across nodes and re-replicated on failure.

Drawbacks of the Report:

- **Small Files Problem**: Very many small files strain NameNode memory; our processor therefore writes micro-batches (every five seconds or 200 records) in time-partitioned folders, and compaction is a future task.

- **Metadata Bottleneck**: The original single NameNode is a scalability and availability concern.

6. **Discretized Streams: Fault-Tolerant Streaming Computation at Scale | Authors: Matei Zaharia, Tathagata Das, Haoyuan Li, Timothy Hunter, Scott Shenker, Ion Stoica | Year: 2013 | Venue: Proceedings of ACM SOSP 2013, pp. 423-438 | Affiliation: University of California, Berkeley, USA**

   Abstract: The paper proposes discretized streams (D-Streams), a micro-batch model in which a stream is treated as a series of short deterministic batch computations, providing fault recovery and second-scale latency.

   Key Points:

   - **Micro-Batch Model**: Splitting the stream into small intervals gives simple fault tolerance and consistency.

   - **Efficient Recovery**: Lost state is recomputed in parallel from lineage instead of by replication.

   - **Unified Analytics**: The same code can be used for streaming and batch computation.

Drawbacks of the Report:

- **Latency Floor**: Micro-batching cannot match record-at-a-time engines in latency.

- **Heavy Infrastructure**: A Spark cluster is more than a single-machine academic project needs; we adopt the micro-batch idea in a lightweight Python processor and note Spark Structured Streaming as an extension.

7. **BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding | Authors: Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova | Year: 2019 | Venue: Proceedings of NAACL-HLT 2019, pp. 4171-4186 | Affiliation: Google AI Language, USA**

   Abstract: The paper introduces BERT, a pre-trained bidirectional transformer that is fine-tuned for downstream tasks and sets state-of-the-art results across many language-understanding benchmarks, including sentiment analysis.

   Key Points:

   - **Transfer Learning**: A model pre-trained on large unlabelled text is fine-tuned on a small labelled set.
   - **Higher Accuracy**: Follow-up work reports accuracies in the mid-90s on IMDB, above classical linear models.
   - **Contextual Representations**: Word meaning depends on the surrounding sentence.

Drawbacks of the Report:

- **Computational Cost**: Training and inference typically need GPUs and have higher latency per review.
- **Interpretability**: Predictions are harder to explain than coefficients of a linear model, which our dashboard uses to show the words that drive each score.

Summary of the Survey and Research Gap:

The surveyed work shows that (i) linear models over weighted n-gram features are accurate, fast and interpretable for movie-review polarity, (ii) Kafka and HDFS are proven building blocks for ingesting and storing event data at scale, and (iii) micro-batch processing offers a simple route to fault-tolerant streaming analytics. Earlier studies of streaming sentiment, such as work on Twitter data streams by Bifet and Frank (2010), concentrate on the learning algorithm. Few studies deliver a complete, reproducible pipeline that connects message streaming, distributed storage, an explainable classifier and a per-title running sentiment

score with an interactive dashboard. This project fills that gap.

Chapter 3

SYSTEM DESIGN & METHODOLOGY

1. Problem Statement and Objectives

- **Problem**: Studios and streaming platforms want near-real-time audience reaction after a release, but current sentiment reporting is manual and delayed by days.
- **Objective**: Stream movie reviews via Kafka, store them on HDFS, train a logistic regression classifier on TF-IDF features to score reviews as positive or negative, and aggregate a running average sentiment score per movie title.
- **Deliverable**: A per-title sentiment dashboard backed by the streaming pipeline.

2. Datasets Used

- **IMDB Dataset of 50K Movie Reviews**: 25,000 positive and 25,000 negative reviews, used to train and evaluate the classifier and to replay a live review stream.
- **Movie Poster Archive**: 8,252 poster images named by IMDb score and IMDb identifier, used for the movie library.
- **IMDb Public Datasets**: The title.basics and title.ratings tables resolve each identifier to a title, year, genre, runtime and vote count; 8,238 posters were matched.

3. System Architecture Overview

- **Speed Layer**: Producer, Kafka and the stream processor score each review as it arrives and update the running aggregates.
- **Batch Layer**: HDFS keeps the historical corpus, the model artefacts and every raw and scored review for reprocessing and retraining.
- **Serving Layer**: A SQLite store and a FastAPI service expose aggregates to the web dashboard. This follows the Lambda architecture pattern of combining batch and speed layers.

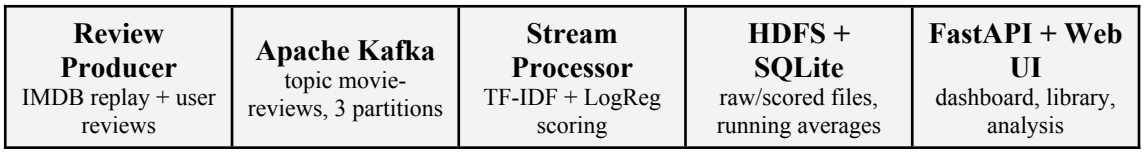


Fig 3.1 : System architecture of CineSentiment

4. Detailed System Components

4.1. Review Producer

- **Function**: Replays the IMDB dataset as a live stream at a configurable rate (1-30 reviews per second).
- **Message Format**: Each message carries an identifier, timestamp, movie title,

review text and label.

- **User Reviews**: The API publishes reviews written in the web interface through the same topic.

4.2. **Apache Kafka**

- **Deployment**: Single-node Kafka 3.7.0 in KRaft mode (no ZooKeeper) with the topic movie-reviews.
- **Partitioning**: Three partitions keyed by movie title, so that all reviews of a title stay ordered.
- **Consumption**: The processor uses the consumer group sentiment-processor with manual offset commits.

4.3. **Stream Processor**

- **Micro-Batches**: Polls up to 200 messages, cleans and vectorises the text, and predicts the probability of positive sentiment for the whole batch.
- **Running Average**: Updates the count, sum of scores and positive count of each title incrementally.
- **Delivery Semantics**: Offsets are committed only after the HDFS write, giving at-least-once delivery; inserts are idempotent on the review identifier.

4.4. **HDFS Storage Layer**

- **Cluster**: One NameNode and one DataNode (Hadoop 3.2.1), replication factor 1, accessed over WebHDFS.
- **Layout**: /imdb/dataset, /imdb/model and /imdb/stream/{raw,scored}/dt=DATE/hr=HOUR/part-*.jsonl.
- **Flush Policy**: A part file is written every five seconds or every 200 records.

4.5. **Sentiment Model**

- **Features**: TF-IDF vectors with a vocabulary of the 5,000 most frequent terms.
- **Classifier**: Logistic regression; the probability of the positive class is the sentiment score (0-100).
- **Explainability**: The product of TF-IDF weight and coefficient shows which words drove a score.

4.6. **Serving Layer and API**

- **Storage**: SQLite tables for reviews, per-title statistics (movie_stats) and pipeline metadata.
- **REST API**: Endpoints for summary, movies, history, recent reviews, pipeline status, prediction, review submission, stream control and model word weights.

4.7. **Web Interface**

- **Home**: Animated poster marquee, live headline numbers and rails of streamed and top-rated titles.
- **Library**: Search, genre filters and sorting over 8,238 posters; a detail view shows IMDb data and live sentiment.
- **Write a Review**: Choose any movie, write a review, see the verdict and the words behind it.
- **Live Dashboard and Analysis**: Pipeline health, per-title running averages, model metrics, confusion matrix, influential words and an IMDb-versus-live-sentiment scatter plot.

5. **Data Preprocessing and Model Training**

- **Cleaning**: URLs are removed, non-alphabetic characters are stripped and text is lower-cased; the same function is used at training and at inference time.
- **Split**: 80% training (40,000 reviews) and 20% test (10,000 reviews) with a fixed random seed of 42.
- **Hyper-parameter Search**: Randomised search over C, solver and maximum iterations, ten candidates with three-fold cross-validation scored on accuracy; the best model uses C = 1.62 and the newton-cg solver.
- **Artefacts**: The model, the TF-IDF vectorizer and a metrics file are saved and archived on HDFS.

6. **Mocking and Simulation Strategy**

- **Movie Titles**: The review dataset has no movie titles, so reviews are assigned to 24 titles chosen from the poster archive, four from each IMDb score band.
- **Audience Positivity**: The chance that a streamed review is drawn from the positive class is derived from the title's IMDb score, so live sentiment tracks, but does not copy, the real rating.
- **Mock Mode**: Setting KAFKA_BOOTSTRAP to mock and STORAGE to local replaces Kafka with a file-backed log and HDFS with a local directory, so the pipeline runs without Docker.

7. **Deployment**

- **Docker Compose**: Services kafka, namenode, datanode, hdfs-ingest, producer, processor, api and frontend start together on one network.
- **Web Server**: nginx serves the static interface, proxies /api to FastAPI and serves the posters.

8. Evaluation Metrics

- **Classification**: Accuracy, precision, recall, F1 score, confusion matrix and ROC-AUC on the test set.
- **Streaming**: Live accuracy against the true labels of streamed reviews, reviews per minute, Kafka consumer lag and the number of HDFS records written.
- **Agreement**: Pearson correlation between the live sentiment of a title and its IMDb rating.

METHODOLOGY

We developed the platform in incremental stages, beginning with the model and ending with the integrated pipeline. First, the IMDB corpus was cleaned and converted into TF-IDF features, and a logistic regression classifier was tuned with randomised search and evaluated on a held-out test set. Second, the streaming path was built: a producer publishes reviews to Kafka, and a processor consumes them in micro-batches, scores them with the saved model, writes raw and scored records to HDFS and updates the running average of each movie title. Third, a FastAPI service and a multi-page web interface were added to present the aggregates, the model analysis and the pipeline status, and to accept reviews written by users.

Reliability and reproducibility guided the design. Offsets are committed only after data reaches HDFS, and database inserts are idempotent, so a restart neither loses nor double-counts reviews. Every component runs in a container and is configured through environment variables, which lets the same code run against real Kafka and HDFS or against their local mocks. The mock catalogue, the audience positivity derived from IMDb scores and the adjustable stream rate allow the behaviour of the system to be demonstrated and tested without access to a live review feed.

Chapter 4

SNAPSHOTS AND RESULTS

[Insert screenshot: Home page with the poster marquee and live statistics]

Fig 1 : Home page with the poster marquee and live statistics

[Insert screenshot: Movie library with search, genre filters and sentiment rings]

Fig 2 : Movie library with search, genre filters and sentiment rings

[Insert screenshot: Movie detail view with live sentiment and trend chart]

Fig 3 : Movie detail view with live sentiment and trend chart

[Insert screenshot: Write a Review page showing the verdict and influential words]

Fig 4 : Write a Review page showing the verdict and influential words

[Insert screenshot: Live dashboard with pipeline status and per-title running averages]

Fig 5 : Live dashboard with pipeline status and per-title running averages

[Insert screenshot: Analysis page with confusion matrix and IMDb-versus-live scatter plot]

Fig 6 : Analysis page with confusion matrix and IMDb-versus-live scatter plot

[Insert screenshot: HDFS NameNode web interface / directory listing of the stream part files]

Fig 7 : HDFS NameNode web interface / directory listing of the stream part files

[Insert screenshot: Docker Compose services running (Kafka, HDFS, processor, API)]

Fig 8 : Docker Compose services running (Kafka, HDFS, processor, API)

[Insert results: classification metrics table, confusion matrix and per-title running averages]

Table 4.1 : Results (to be added)

Chapter 5

CONCLUSION
FUTURE ENHANCEMENTS

In summary, CineSentiment demonstrates that near-real-time audience sentiment can be obtained with a compact big data pipeline. Reviews flow through Kafka, are persisted on HDFS, and are scored by a TF-IDF and logistic regression classifier that reaches 89.39% accuracy and a ROC-AUC of 0.9606 on 10,000 held-out reviews. A running average per movie title replaces the manual, delayed reporting described in the problem statement, and the web interface makes the pipeline, the model and the audience reaction visible to a non-technical user, including through reviews that users write themselves.

Several improvements could extend the platform. The stream processor can be moved to Spark Structured Streaming or Apache Flink for horizontal scaling, and Kafka and HDFS can be deployed as multi-node clusters with replication factor three. Small part files on HDFS can be compacted into columnar Parquet files, and the stored history can be used for scheduled batch retraining and drift monitoring. Transformer models such as DistilBERT could raise accuracy at the cost of latency, and aspect-based sentiment would reveal what viewers like or dislike about acting, plot or effects.

Further work includes mapping real review feeds to IMDb identifiers instead of the simulated assignment, windowed trend alerts that notify a studio when sentiment for a title drops sharply, handling of sarcasm and multilingual reviews, authentication and rate limiting on the review API, and deployment on Kubernetes with monitoring of consumer lag, throughput and model quality.

REFERENCES

- Pang, B., Lee, L., & Vaithyanathan, S. (2002). Thumbs up? Sentiment classification using machine learning techniques. In Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 79-86.

- Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y., & Potts, C. (2011). Learning word vectors for sentiment analysis. In Proceedings of ACL-HLT 2011, pp. 142-150.

- Wang, S., & Manning, C. D. (2012). Baselines and bigrams: Simple, good sentiment and topic classification. In Proceedings of ACL 2012 (Short Papers), pp. 90-94.

- Kreps, J., Narkhede, N., & Rao, J. (2011). Kafka: A distributed messaging system for log processing. In Proceedings of the NetDB Workshop.

- Shvachko, K., Kuang, H., Radia, S., & Chansler, R. (2010). The Hadoop Distributed File System. In IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST), pp. 1-10.

- Zaharia, M., Das, T., Li, H., Hunter, T., Shenker, S., & Stoica, I. (2013). Discretized streams: Fault-tolerant streaming computation at scale. In Proceedings of ACM SOSP 2013, pp. 423-438.

- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In Proceedings of NAACL-HLT 2019, pp. 4171-4186.

- Bifet, A., & Frank, E. (2010). Sentiment knowledge discovery in Twitter streaming data. In Discovery Science (DS 2010), LNCS 6332, pp. 1-15.

- Socher, R., et al. (2013). Recursive deep models for semantic compositionality over a sentiment treebank. In Proceedings of EMNLP 2013, pp. 1631-1642.

- Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

- Ramos, J. (2003). Using TF-IDF to determine word relevance in document queries. In

Proceedings of the First Instructional Conference on Machine Learning.

- Liu, B. (2012). Sentiment Analysis and Opinion Mining. Morgan & Claypool Publishers.

- Marz, N., & Warren, J. (2015). Big Data: Principles and Best Practices of Scalable Real-Time Data Systems. Manning Publications.

- Pathi, L. N. IMDB Dataset of 50K Movie Reviews (Kaggle). Retrieved from https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews

- IMDb Non-Commercial Datasets (title.basics, title.ratings). Retrieved from https://developer.imdb.com/non-commercial-datasets/

- Apache Kafka Documentation. Retrieved from https://kafka.apache.org/documentation/

- Apache Hadoop HDFS Architecture. Retrieved from https://hadoop.apache.org/docs/r3.2.1/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html

- himarygr. IMDB-Review-Classifier (open-source baseline of the TF-IDF and logistic regression classifier, extended in this project). Retrieved from https://github.com/himarygr/IMDB-Review-Classifier

APPENDIX

Technology versions:

Component	Version / Image
Apache Kafka (KRaft mode)	apache/kafka 3.7.0
Hadoop HDFS (NameNode + DataNode)	bde2020 images, Hadoop 3.2.1
Python / scikit-learn	3.9 / 1.4.2
API / web server	FastAPI with Uvicorn / nginx (alpine)
Orchestration	Docker Compose

REST API endpoints:

Method and path	Purpose
GET /api/summary	Totals, average sentiment, positive share, live accuracy
GET /api/movies	Running average, count and trend per title
GET /api/movies/history	Running average after each review (one title)
GET /api/recent	Latest scored reviews
GET /api/pipeline	Kafka, processor and HDFS status and model metrics
POST /api/predict	Score a review and return influential words
POST /api/review	Score a user review and publish it to Kafka
POST /api/control	Change stream rate or pause the producer
GET /api/model/words	Most positive and negative TF-IDF terms

How to run:

cd docker && docker compose up --build -d

Dashboard: http://localhost:8501 API docs: http://localhost:8000/docs NameNode UI: http://localhost:9870

PROJECT SOURCE CODE / LINK:

[Add the project repository link here]